

STRUCTURED QUERY LANGUAGE

INTRODUCTION TO DATA

DATA:

Data is a raw fact which describes the attributes of an entity.

EX: Entity is SMARTPHONE

ATTRIBUTES/PROPERTIES	DATA/VALUE
Brand	iPhone 14
Price	65999/-
RAM	6GB
Storage	128GB
Camera	64MP
Battery Capacity	3256mAH

EX: Entity is HUMAN

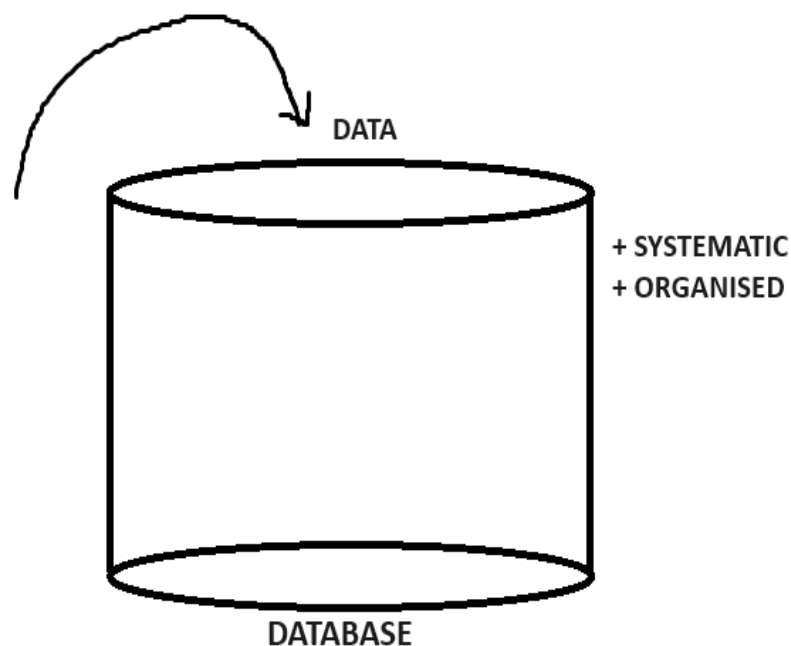
ATTRIBUTES/PROPERTIES	DATA/VALUE
Name	Rama
Place	Bengaluru
Contact No	9876543210
Email ID	Rama1@gmail.com
Aadhaar No	111122223333
Qualification	BE(EEE)

WHAT IS DATABASE?

DATABASE:

- Database is a place or medium where the data is stored in systematic and organized manner.
- We can perform certain operations on the database such as
 - CREATE / INSERT
 - READ / RETRIEVE
 - UPDATE / MODIFY
 - DROP / DELETE
- These operations are universally referred as CRUD operations.

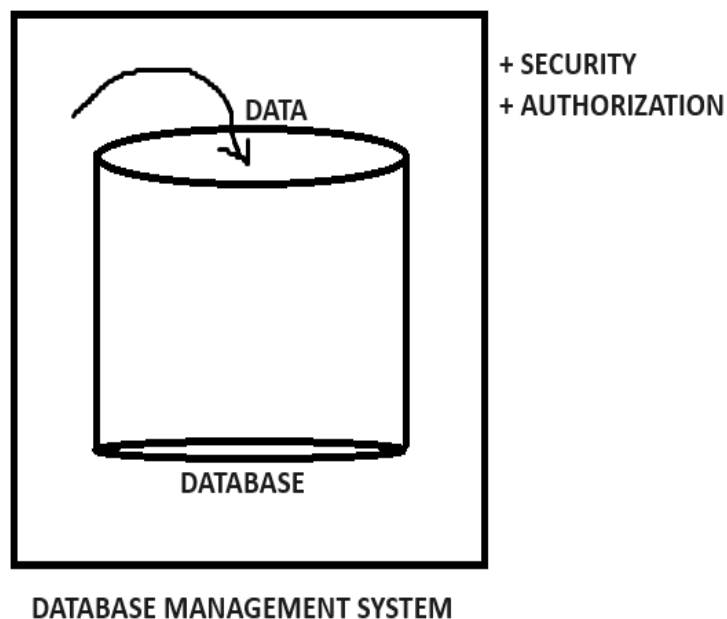
EX: Contacts, Facebook , Zomato , Amazon , Swiggy



DATABASE MANAGEMENT SYSTEM

DBMS:

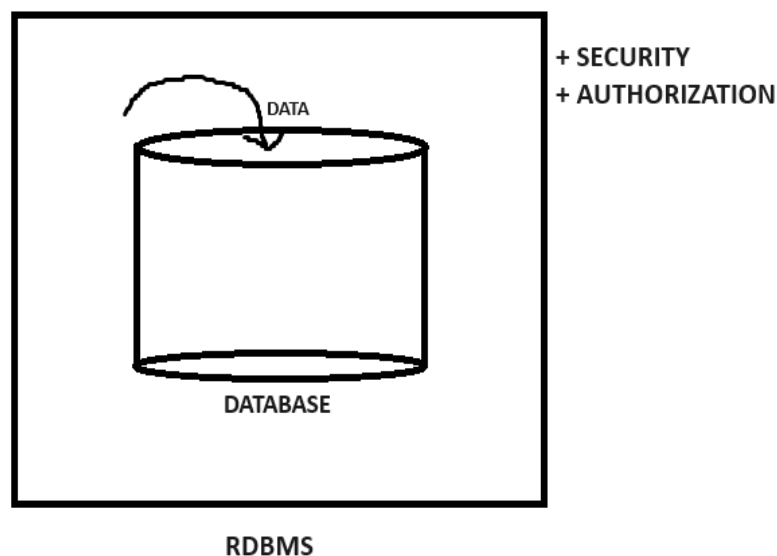
- DBMS is a software which is used to **maintain** and **manage** database.
- DBMS provides two main features **Security** and **Authorization**.
- In DBMS the data is stored in the form of **files**.
- We need **Query language** to communicate with DBMS.



RELATIONAL DATABASE MANAGEMENT SYSTEM

RDBMS:

- RDBMS is a software which is used to **maintain** and **manage** database.
- RDBMS provides two main features **Security** and **Authorization**.
- In RDBMS the data is stored in the form of **Tables**.
- We need **structured query language** to communicate with RDBMS.

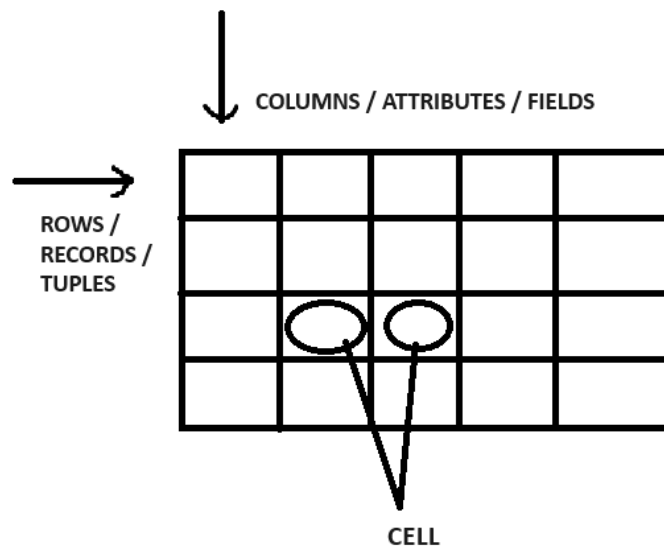


RELATIONAL MODEL:

- Relational model was given by E F CODD.
- Any DBMS which follows relational model will become RDBMS.

TABLE:

- Table is the logical organization of data in the form of rows and columns.
- Cell is the smallest unit of a table.



RULES OF E. F. CODD

RULE 1:

- The data entered into a cell must be **single valued**.
- The data must be single valued **to avoid data loss**.

RULE 2:

- In RDBMS everything will be stored in the form of tables including **metadata**.
- **Metadata** means details about the data.
- **Metadata** will be stored in **Meta table**.

- **Meta table** will be automatically generated by software.

RULE 3:

- According to E. F. CODD we can store the data in multiple tables, if needed we can establish the relation between the tables by using **Key attributes**.

RULE 4:

- We can validate the data entered into the tables by **two** steps,
 - By assigning **Datatypes**
 - By assigning **Constraints**

NOTE: DATATYPES ARE MANDATORY, CONSTRAINTS ARE OPTIONAL.

DATAYPES

Datatypes are used to determine what kind or type of data should be stored in a particular memory location.

TYPES OF DATATYPES:

- 1) CHAR
- 2) VARCHAR / VARCHAR (2)
- 3) DATE
- 4) NUMBER
- 5) LARGE OBJECTS

- Character Large Objects (CLOB)
- Binary Large Objects (BLOB)

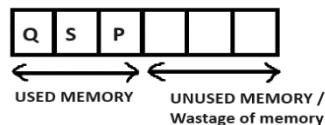
1) CHAR:

- CHAR accepts 'A-Z', 'a-z', '0-9', special characters and alphanumeric characters.
- Whenever we pass CHAR datatype, we must mention size for it.

SYNTAX: CHAR(SIZE)

- The characters must be enclosed within **single quotes**.
- Maximum characters can be passed is 2000.
- CHAR follows **Fixed Length Memory Allocation**.

EX: CHAR (6)



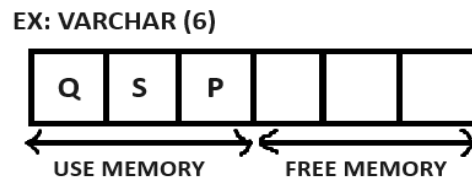
2) VARCHAR / VARCHAR (2):

- VARCHAR accepts 'A-Z', 'a-z', '0-9', special characters and alphanumeric characters.
- Whenever we pass VARCHAR datatype, we must mention size for it.

SYNTAX: VARCHAR (SIZE)

- The characters must be enclosed within **single quotes**.
- Maximum characters can be passed is **2000**.

- VARCHAR follows **Variable Length Memory Allocation**.



NOTE:

- a) VARCHAR2 is updated version of VARCHAR.
- b) VARCHAR2 accepts maximum characters up to **4000**.

3) DATE:

- Date datatype is used to store only dates in a specified format.
- We follow **Oracle Specified Format**.
- The formats are,
 - **‘DD-MON-YYYY’** or **‘DD-MON-YY’**
- Dates must be enclosed within single quotes.
-

4) NUMBER:

- It is used to store only numerical values.
- **SYNTAX: NUMBER (PRECISION, [SCALE])**

PRECISION:

- It is the total number of integer digits stored in a value.
- The range of precision is **1 to 38**.
- **EX: NUMBER (4): +/- 9999**
- **EX: NUMBER (6): +/- 999999**
- **EX: NUMBER (2,0): +/- 99**

SCALE:

- It is the total number of decimal digits stored in a value within precision.
- The range of scale is **-84 to 127**.
- **EX: NUMBER (4,2): +/- 99.99**
- **EX: NUMBER (5,3): +/- 99.999**
- **EX: NUMBER (7,2): +/- 99999.99**
- **EX: NUMBER (3,5): +/- .00999**
- **EX: NUMBER (4,8): +/- .00009999**

5) LARGE OBJECTS:

a) CHARACTER LARGE OBJECTS (CLOB):

- It is used to store large number of characters up to 4GB of size.

b) BINARY LARGE OBJECTS (BLOB):

- It is used to store binary values of images, mp3, mp4, documents etc up to 4GB of size.

CONSTRAINTS

Constraints are the rules that are assigned to the columns for validating the data.

TYPES OF CONSTRAINTS:

- 1) UNIQUE
- 2) NOT NULL
- 3) CHECK
- 4) PRIMARY KEY
- 5) FOREIGN KEY

1) UNIQUE:

Unique is the constraint assigned to column which is used to avoid duplicate or repeated values.

2) NOT NULL:

- NOT NULL is the constraint which is assigned to column to avoid null.
- NULL means empty or nothing.
- ZERO is not a null because it holds memory.
- Any arithmetic operations performed with null will result into null.

3) CHECK:

- CHECK is a constraint assigned to column for extra validation.
- CHECK will be under condition, if the condition is satisfied the value will be accepted else the value will be rejected.
- EX: CHECK (SAL>0)

- EX: CHECK(LENGTH(CNO)=10)

4) PRIMARY KEY:

Primary key is a constraint which is used to uniquely identify a record from the table.

CHARACTERISTICS OF PRIMARY KEY:

- Primary key will not accept repeated values or duplicate values.
- Primary key will not accept null.
- Primary key is the combination of unique and not null.
- We can have only one primary key in a table.
- Primary key is not mandatory, but it is highly recommended to have one in a table.

5) FOREIGN KEY:

Foreign key is used to establish relation between the tables.

CHARACTERISTICS OF FOREIGN KEY:

- Foreign key will accept duplicate or repeated values.
- Foreign key will accept null.
- Foreign key is not a combination of unique and not null.
- We can have any number of foreign keys in a table.
- To become a foreign key, it should be primary key in its own table.
- Foreign keys are actually present in child table, but they actually belong to parent table.
- Foreign key is also called as **Referential Integrity Constraint**.

TABLE NAME: EMP

EMPNO	ENAME	SAL	CNO
CHAR (3)	VARCHAR (25)	NUMBER (7,2)	NUMBER (10)
UNIQUE/NOT NULL/ P. K	NOT NULL	NOT NULL / CHECK (SAL>0)	UNIQUE/NOT NULL / CHECK (LENGTH(CNO)=10)
A01	SANJAY	9000	9876543210
A02	ADAMS	10000	9876543211
A03	MILLER	8000	9876543122
A04	JAMES	7500	9879879870
A05	SCOTT	6400	8989076543

OVERVIEW OF SQL STATEMENTS

- 1) DATA DEFINITION LANGUAGE (DDL)
- 2) DATA MANIPULATION LANGUAGE (DML)
- 3) TRANSACTION CONTROL LANGUAGE (TCL)
- 4) DATA CONTROL LANGUAGE (DCL)
- 5) DATA QUERY LANGUAGE (DQL)

DATA QUERY LANGUAGE

This language is used to retrieve the data from the database.

It has 4 statements,

1) SELECT:

Select is used to retrieve the data from the database and display it.

2) PROJECTION:

Projection is used to retrieve the data from the table by selecting only the columns.

3) SELECTION:

Selection is used to retrieve the data from the table by selecting both rows and columns.

4) JOIN:

The process of retrieving the data from multiple tables simultaneously is known as joins.

PROJECTION

Projection is used to retrieve the data from the table by selecting only the columns.

SYNTAX:

```
SELECT */ [DISTINCT] Column name / Expression [ALIAS]  
FROM Table name;
```

Note:

- From clause starts the execution.
- For From clause, we can pass Table Name as an argument (Input).
- The job of From clause is to go to the database search for the table and put the table under execution.
- Select clause executes after the execution of From clause.
- For select clause we can pass 3 arguments
 - Asterisk (*)
 - Column Name
 - Expression
- The job of select clause is to go to the Table under execution and display the columns mentioned.
- Select clause is responsible for preparing the Result Table
- Semi Colon (;): It is used to represent end of the Query.
- Asterisk (*): It means to select all the columns from the table
-

DISTINCT CLAUSE:

- Distinct clause is used to remove the duplicate or repeated values from the table.
- Distinct must be the first argument in select clause.
- We can pass multiple arguments / column names in distinct clause.
- EX: SELECT DISTINCT SAL
FROM EMP;

EXPRESSION:

Any statement which gives result is known as expression.

Expression consists of **Operands** and **Operator**.

Operands: These are the values that user passes.

Operands can be of

- CHARACTER LITERAL
- DATE LITERAL
- NUMBER LITERAL

CHARACTER and DATE literals must be enclosed within single quotes.

Operator: It is the symbol operated between the operands.

EX: SELECT SAL*12

FROM EMP;

HERE SAL*12 is expression,

SAL and 12 are operands and (*) is operator.

ALIAS:

- It is the alternate name given to the column in result table.
- Alias names can be used with or without using 'AS' keyword.
- EX:

```
SELECT SAL*12 AS Ann_sal  
FROM EMP;
```
- EX:

```
SELECT SAL+SAL*10/100 AS HIKE  
FROM EMP;
```
- EX:

```
SELECT SAL*12 "ANN_SAL"  
FROM EMP;
```


SELECTION

Selection is used to retrieve the data from the table by selecting both rows and columns.

SYNTAX:

```
SELECT */ [DISTINCT] Column name / Expression [ALIAS]
FROM Table name;
WHERE <Filter condition>;
```

ORDER OF EXECUTION:

1 – FROM

2 – WHERE

WHERE CLAUSE:

- WHERE clause is used to filter the records.
- WHERE clause executes row by row.
- WHERE clause executes after from clause.
- In WHERE clause we pass filter conditions.
- WHERE clause gives output in Boolean form (TRUE/FALSE).
- In WHERE clause we can pass multiple filter conditions by using **logical operators**.

OPERATORS IN SQL

- 1) ARITHMETIC OPERATORS: (+, -, *, /)
- 2) CONCATENATION OPERATOR: (||)
- 3) COMPARISON OPERATORS: (=, !=)
- 4) RELATIONAL OPERATOR: (>, <, >=, <=)
- 5) LOGICAL OPERATORS: (AND, OR, NOT)
- 6) SPECIAL OPERATORS:
(IN, NOT IN, BETWEEN, NOT BETWEEN, IS, IS NOT, LIKE, NOT LIKE)
- 7) SUBQUERY OPERATORS:
(ALL, ANY, EXISTS, NOT EXISTS)

LOGICAL OPERATORS:

1) AND OPERATOR:

AND operator is used to satisfy multiple conditions.

EX: WAQTD details of emps working as manager in dept 20

```
SELECT *  
FROM EMP  
WHERE JOB = 'MANAGER' AND DEPTNO = 20 ;
```

2) OR OPERATOR:

OR operator is used to satisfy at least one condition.

EX: WAQTD details of emps working in dept 20, 30

```
SELECT *  
FROM EMP  
WHERE DEPTNO = 20 OR DEPTNO = 30;
```

3) NOT OPERATOR:

NOT operator is used to reject the given condition.

EX: WAQTD details of emps except salesman.

```
SELECT *  
FROM EMP  
WHERE JOB != 'SALESMAN';
```

(OR)

```
SELECT *  
FROM EMP  
WHERE NOT JOB = 'SALESMAN';
```

1) IN OPERATOR:

- IN is a multivalued operator which accept single value at LHS and multiple values at RHS.
- IN returns true if any one of the conditions at the RHS is satisfied.

SYNTAX: Column name / expression IN (V1, V2, V3.....Vn);

EX:WAQTD names of the employees working in dept 10 or 20 or 30

```
SELECT ENAME  
FROM EMP  
WHERE DEPTNO IN (10, 20, 30);
```

2) NOT IN OPERATOR:

- NOT IN is same as IN operator instead of accepting the values it will reject the values.

SYNTAX: Column name / expression NOT IN (V1, V2, V3.....Vn);

EX: WAQTD details of emps except manager and clerk.

```
SELECT *  
FROM EMP  
WHERE JOB NOT IN ('MANAGER', 'CLERK');
```

3) BETWEEN OPERATOR:

- BETWEEN operator is used whenever we have the range of values.
- BETWEEN operator includes both the given ranges.

SYNTAX:

Column name / expression BETWEEN lower range AND higher range;

EX: WAQTD name and sal of emps earning sal in the range of 1100 and 3000.

```
SELECT ENAME , SAL  
FROM EMP  
WHERE SAL BETWEEN 1100 AND 3000;
```

4) NOT BETWEEN OPERATOR:

- NOT BETWEEN is same as BETWEEN instead of accepting the range it will reject the range.

SYNTAX:

Column name / expression NOT BETWEEN lower range AND higher range;

```
SELECT *  
FROM EMP  
WHERE SAL NOT BETWEEN 1100 AND 3000;
```

5) IS OPERATOR:

- IS operator is used to compare only with NULL.

SYNTAX: Column name / expression IS NULL;

EX: WAQTD details of emps who do not earn any commission.

```
SELECT *  
FROM EMP  
WHERE COMM IS NULL;
```

6) IS NOT OPERATOR:

- IS NOT operator is used to compare with not null values.

SYNTAX: Column name / expression IS NOT NULL;

EX: WAQTD details of emps who do earn some commission.

```
SELECT *  
FROM EMP  
WHERE COMM IS NOT NULL;
```

7) LIKE OPERATOR:

- LIKE operator is used for pattern matching.
- To achieve the pattern matching we use special operators like **PERCENTILE (%)** and **UNDERSCORE (_)**.
- **PERCENTILE:** It will accept any number of characters any number of times and no character.
- **UNDERSCORE:** It will accept only one character but any character.

SYNTAX: Column name / expression LIKE 'pattern_to_match';

EX: WAQTD names of emps whose name starts with 'A'.

```
SELECT ENAME  
FROM EMP  
WHERE ENAME LIKE 'A%';
```

8) NOT LIKT OPERATOR:

- NOT LIKE is same as LIKE operator instead of accepting the values it will reject the values.

SYNTAX:

Column name / expression NOT LIKE 'pattern_to_match';

EX: WAQTD names of emps whose name does not start with 'A'.

```
SELECT ENAME  
FROM EMP  
WHERE ENAME NOT LIKE 'A%';
```

FUNCTIONS

Functions are block of code or list of instructions to perform a specific task.

Components of functions are,

- 1) Function name
- 2) Number of Arguments
- 3) Return Type

TYPES OF FUNCTIONS:

- 1) In-built functions
- 2) User defined functions

In In-built functions we have again two types

- 1) Single Row Functions
- 2) Multi Row Functions

SINGLE ROW FUNCTIONS

- Single Row Functions executes row by row.
- Single Row Functions takes one input process and produces one output.
- If we pass N number of inputs, we will get N number of outputs.



EX: SELECT LENGTH(ENAME)
FROM EMP;

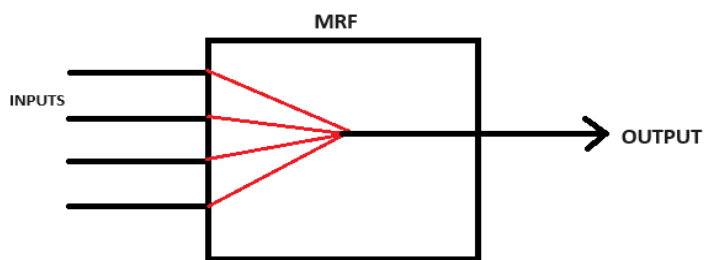
ENAME	SAL
SMITH	2000
ALLEN	2500
WARD	3500
MILLER	4500



LENGTH(ENAME)
5
5
4
6

MULTI ROW FUNCTIONS

- Multirow functions executes group by group.
- Multirow functions takes N number of inputs process and gives exactly one output.
- If we pass N number of inputs, we will get single output.



EX: SELECT MAX(SAL)
FROM EMP;

ENAME	SAL
SMITH	2000
ALLEN	2500
WARD	3500
MILLER	4500



MAX(SAL)
4500

TYPES OF MULTIROW FUNCTIONS

- 1) MAX()
- 2) MIN()
- 3) AVG()
- 4) SUM()
- 5) COUNT()

GROUP BY CLAUSE

- Group by clause is used to group the records.
- Group by clause executes row by row.
- Group by clause executes after where clause (if present).
- We can use Group by clause without using WHERE clause.
- We can pass column name or expression in group by clause.
- After the execution of group by clause we get groups.
- Any clause which executes after group by clause will execute in groups.

SYNTAX:

SELECT GROUP_BY_EXPRESSION / GROUP FUNCTION

FROM TABLE_NAME

[WHERE <FILTER_CONDITION>]

GROUP BY COL_NAME / EXP;

ORDER OF EXECUTION:

1 – FROM

2- WHERE (if present) [ROW-BY-ROW]

3-GROUP BY [ROW-BY-ROW]

4- SELECT [GROUP-BY-GROUP]

HAVING CLAUSE

- HAVING clause is used to filter the groups.
- HAVING clause executes group by group.
- HAVING clause executes after group by clause.
- We cannot use HAVING clause without using GROUP BY clause.
- In HAVING clause, we pass group filter conditions.
- We can pass multirow functions in HAVING clause.

SYNTAX:

SELECT GROUP_BY_EXPRESSION / GROUP FUNCTION

FROM TABLE_NAME

[WHERE <FILTER_CONDITION>]

GROUP BY COL_NAME / EXP

HAVING <group_filter_condition>;

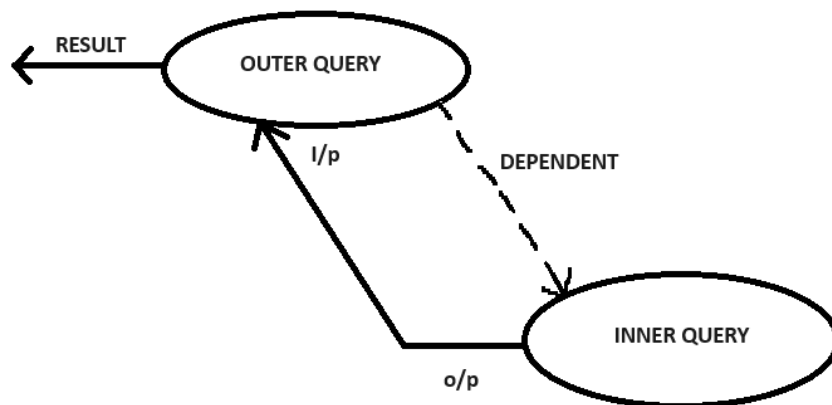
ORDER OF EXECUTION:

- 1- FROM
- 2- WHERE (if used) [ROW-BY-ROW]
- 3- GROUP BY [ROW-BY-ROW]
- 4- HAVING [group-by-group]
- 5- SELECT [group-by-group]

SUBQUERY

A query written inside another query is known as subquery.

WORKING PRINCIPLE:



Let us consider 2 queries inner query and outer query,

- Inner query executes first and gives one output.
- Output of inner query will be passed as an input to outer query.
- Outer query executes and gives result.
- By this we can say outer query is dependent on inner query.

This is the working principle of subquery.

WHEN / WHY DO WE USE SUBQUERY:

CASE1: Whenever we have unknown or indirect conditions present in the question, we use subquery.

```
EX: SELECT ENAME  
      FROM EMP  
      WHERE SAL > 2000;
```

In above example SAL is compared to 2000 which is a direct condition.

EX: WAQTD names of emps earning more than ALLEN.

```
SELECT ENAME  
FROM EMP  
WHERE SAL > (SELECT SAL  
              FROM EMP  
              WHERE ENAME = 'ALLEN');
```

In above example salary is compared with a name which is known as indirect or unknown condition. Here we have to use subquery.

CASE2: Whenever the data to be selected and the condition to be executed are present in different tables than we use subquery.

EX: WAQTD deptno of james.

```
SELECT DEPTNO  
FROM EMP  
WHERE ENAME = 'JAMES';
```

In above example the data to be selected (deptno) and condition to be executed (james), both are present in emp table so we can use normal query.

EX: WAQTD dname of JAMES.

```
SELECT DNAME
FROM DEPT
WHERE DEPTNO = (SELECT DEPTNO
                  FROM EMP
                  WHERE ENAME = 'JAMES');
```

TYPES OF SUBQUERY:

- 1) Single Row Subquery
- 2) Multi Row Subquery

1) Single Row Subquery:

- When subquery returns exactly one row or one record is called as single row subquery.
- When subquery returns exactly one row or one record we can use normal operators.

EX: WAQTD details of emps earning salary more than smith

```
SELECT *
FROM EMP
WHERE SAL > (SELECT SAL
              FROM EMP
              WHERE ENAME = 'SMITH');
```

In above example as there is only one smith. So subquery exactly returns only one value so it is called single row subquery.

2) Multi Row Subquery:

- When subquery returns more than one value or one record than it is called multi row subquery.
- When subquery returns more than one record or one value than we cannot use normal operators instead we have to use special operators (ALL, ANY, IN).

EX: WAQTD names of emps earning more than managers.

Here there are more than one manager, so subquery returns more than one value or one record.

```
➤ SELECT ENAME
FROM EMP
WHERE SAL >ALL (SELECT SAL
                FROM EMP
                WHERE JOB = 'MANAGER');
```

```
➤ SELECT ENAME
FROM EMP
WHERE SAL >ANY (SELECT SAL
                FROM EMP
                WHERE JOB = 'MANAGER');
```

SUBQUERY OPERATORS:

1) ALL OPERATOR:

- ALL operator is a special operator which must be used along with the relational operators.

- ALL operator returns true if all the conditions at the RHS is satisfied.

2) ANY OPERATOR:

- ANY operator is a special operator which must be used along with the relational operators.
- ANY operator returns true if any one of the conditions at the RHS is satisfied.

NESTED SUBQUERY:

- A subquery written inside another subquery is known as nested subquery.
- We can nest about 255 subqueries.
- Innermost query executes first.

EX: WAQTD 3rd maximum salary.

```
SELECT MAX(SAL)
FROM EMP
WHERE SAL < (SELECT MAX(SAL)
              FROM EMP
              WHERE SAL < (SELECT MAX(SAL)
                            FROM EMP));
```

NOTE:

- 1) We can only pass one column name or expression in the select clause of inner query.
- 2) The column name in the select clause of inner query and where clause of outer query need not be same but their datatypes must be same.

JOINS

The process of retrieving the data from multiple tables simultaneously is known as joins.

WHEN OR WHY DO WE JOINS:

When we need to display the attributes from multiple tables at the same time, we use concept of joins.

TYPES OF JOINS:

- 1) Cartesian join or cross join.
- 2) Inner join or equi join.
- 3) Outer join
 - Left outer join.
 - Right outer join.
 - Full outer join.
- 4) Natural join.
- 5) Self-join.

1) Cartesian Join/Cross join

- In cartesian join a record from one table will be merged with all the records of another table.
- No of columns in result in table will be equal to summation of no of columns in both the tables.
- No of records in result table will be equal to product of no of records in both the tables.

SYNTAX:

1. ANSI:

SELECT Column Name

FROM Table_Name1 CROSS JOIN Table_Name2;

E.g. SELECT *

FROM EMP CROSS JOIN DEPT;

2. ORACLE:

SELECT Column Name

FROM Table_Name1, Table_Name2 ;

E.g. SELECT *

FROM EMP, DEPT;

2) Inner join/Equi join:

- Inner join is used to obtain only matching records.
- To obtain matching records we need to pass join condition.
- JOIN CONDITION: This is the condition where two tables are merged.
- **JOIN CONDITION = EMP.DEPTNO = DEPT.DEPTNO**

SYNTAX:

1. ANSI :

```
SELECT Column Name  
FROM Table_Name1 INNER JOIN Table_Name2  
ON <JOIN_CONDITION>;
```

EX: SELECT *

```
FROM EMP INNER JOIN DEPT  
ON EMP.DEPTNO = DEPT.DEPTNO;
```

2.ORACLE:

```
SELECT Column Name  
FROM Table_Name1 , Table_Name2  
WHERE <JOIN_CONDITION> ;
```

EX: SELECT *

```
FROM EMP, DEPT  
WHERE EMP.DEPTNO = DEPT.DEPTNO;
```

3) Natural Join:

Natural join has 2 behaviours

- It behaves like Inner join if there is relation between the tables.
- It behaves like cartesian join if there is no relation between the tables.

SYNTAX:

ANSI:

SELECT column Name

FROM Table_Name1 NATURAL JOIN Table Name2;

EX1:

SELECT *

FROM EMP NATURAL JOIN DEPT;

EX2:

SELECT *

FROM EMP NATURAL JOIN CUSTOMER;

4) Outer join:

- Outer join is used to obtain unmatched records along with matching records.

a) LEFT OUTER JOIN:

- Left outer join is used to obtain unmatched records of left table along with matching records.

SYNTAX:

1. ANSI:

```
SELECT Column Name  
  
FROM Table_Name1 LEFT [OUTER] JOIN Table_Name2  
  
ON <JOIN_CONDITION>;
```

EX: SELECT *

```
FROM EMP E LEFT OUTER JOIN DEPT D  
  
ON E. DEPTNO = D. DEPTNO;
```

2.ORACLE:

```
SELECT Column Name  
  
FROM Table_Name1, Table_Name2  
  
WHERE Table_Name1.Col_Name=Table_Name2.Col_Name(+);
```

EX: SELECT *

```
FROM EMP E, DEPT D  
  
WHERE E. DEPTNO = D. DEPTNO (+);
```

b) RIGHT OUTER JOIN:

- Right outer join is used to obtain unmatched records of right table along with matching records.

1. ANSI:

```
SELECT Column Name  
  
FROM Table Name1 RIGHT [OUTER] JOIN Table Name2  
  
ON <JOIN_CONDITION>;
```

EX: SELECT *

```
FROM EMP E RIGHT OUTER JOIN DEPT D  
  
ON E.DEPTNO = D.DEPTNO ;
```

2.ORACLE: SELECT Column Name

```
FROM Table_Name1, Table_Name2  
  
WHERE Table_Name1.Col_Name (+) = Table_Name2.Col_Name;
```

EX: SELECT *

```
FROM EMP E, DEPT D  
  
WHERE E.DEPTNO (+) = D.DEPTNO ;
```

c) FULL OUTER JOIN:

- Full outer join is used to obtain unmatched records of both left and right tables along with matching records.

1. ANSI:

```
SELECT Column Name  
  
FROM Table_Name1 FULL [OUTER] JOIN Table_Name2  
  
ON <JOIN_CONDITION>;
```

EX: SELECT *

```
FROM EMP E FULL OUTER JOIN DEPT D  
  
ON E.DEPTNO = D.DEPTNO ;
```

5) SELF JOIN:

- Joining the same two tables or joining a table by itself is called as self-join.

SYNTAX:

ANSI:

```
SELECT Column Name  
  
FROM Table Name T1 JOIN Table Name T2  
  
ON <Join Condition>;
```

EX:

```
SELECT *  
  
FROM EMP E1 JOIN EMP E2  
  
ON E1.MGR = E2.EMPNO;
```


ORACLE:

SELECT Column Name

FROM Table Name T1, Table Name T2

WHERE < Join Condition>;

EX:

SELECT *

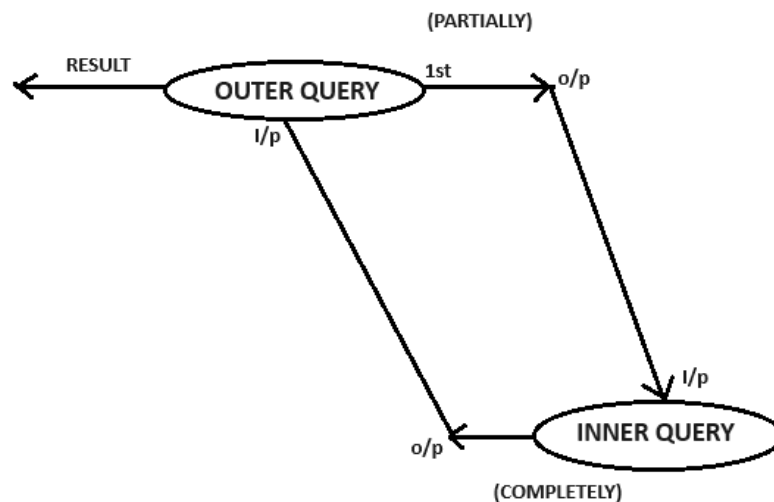
FROM EMP E1, EMP E2

WHERE E1.MGR = E2.EMPNO;

CORRELATED SUBQUERY

A query written inside another query such that both outer and inner query are dependent on each other.

WORKING PRINCIPLE:



Let us consider 2 queries inner query and outer query,

- Outer query executes first but partially.
- Partial output of outer query will be passed as an input to inner query.
- Inner query executes completely and gives one output.
- Output of inner query will be again passed as an input to outer query.
- Outer query executes and gives result.

- By this we can say both outer query and inner query are dependent on each other (Interdependent).

This is the working principle of correlated subquery.

EX: WAQTD dept names in which employees are working.

- SELECT D. DNAME
FROM DEPT D
WHERE D. DEPTNO IN (SELECT E. DEPTNO
FROM EMP E
WHERE D. DEPTNO = E. DEPTNO)

EXISTS OPERATOR:

- EXISTS is a unary operator which accepts single operand towards RHS.
- EXISTS operator returns TRUE if subquery returns a value.
- EXISTS is used to increase the efficiency of correlated subquery.

NOT EXISTS OPERATOR:

- NOT EXISTS is a unary operator which accepts single operand towards RHS.
- NOT EXISTS operator returns TRUE if subquery returns NULL.
- NOT EXISTS is used to increase the efficiency of correlated subquery.

TO FIND Nth MAXIMUM AND Nth MINIMUM SALARY BY USING CORELATED SUBQUERY:

SYNTAX:

TO FIND Nth MAXIMUM SALARY:

```
SELECT E1.SAL
FROM EMP E1
WHERE N-1 IN (SELECT COUNT (DISTINCT E2.SAL)
              FROM EMP E2
              WHERE E1.SAL < E2.SAL);
```

(OR)

```
SELECT E1.SAL
FROM EMP E1
WHERE (SELECT COUNT (DISTINCT E2.SAL)
      FROM EMP E2
      WHERE E1.SAL < E2.SAL) IN (N-1);
```

TO FIND Nth MINIMUM SALARY:

SELECT E1.SAL

FROM EMP E1

WHERE N-1 IN (SELECT COUNT (DISTINCT E2.SAL)

FROM EMP E2

WHERE E1.SAL > E2.SAL);

(OR)

SELECT E1.SAL

FROM EMP E1

WHERE (SELECT COUNT (DISTINCT E2.SAL)

FROM EMP E2

WHERE E1.SAL > E2.SAL) IN (N-1);

